PluggedIn: Representing Electric Vehicle Charging Accessibility*

Jungwon Youn
University of Victoria
Victoria, BC, Canada
jungwonyoun@uvic.ca

Sunghun Cho
University of Victoria
Victoria, BC, Canada
shcho980901@uvic.ca

Elias Dockal
University of Victoria
Victoria, BC, Canada
eliasdockal@uvic.ca

ABSTRACT

EV charging usability depends on factors like connector compatibility, operating hours, access conditions, and network memberships. Comparing existing infrastructure data and semantic models reveals critical representation gaps. To address these, we present PluggedIn, a relational data model centered on vehicle-specific practical usability that integrates direct connector compatibility, driver network memberships, and structured operating periods, laying the groundwork for future knowledge graphs. The project repository and data models are available at https://github.com/chohoon901/PluggedIn.

KEYWORDS

EV charging, relational data modeling, SQLite, connector compatibility, data accessibility

1 PROBLEM DEFINITION

Our goal is to enable drivers and fleet managers to retrieve station availability, connector compatibility, operating hours, and payment conditions. Flat infrastructure lists fail to capture vehicle-to-port suitability, access restrictions, and membership constraints.

We link charging stations to structured operating periods, ports, standardized connector types, and organizational roles, while explicitly associating drivers with network memberships. Our database answers a core operational question: *"Can a specific vehicle charge at this station through a direct connection, and is it open and accessible under the driver's network membership?"*

2 PROPOSED RELATIONAL MODEL

2.1 Station, Organization, and Core Entity Relations

Station records coordinates, address, parking, access and vehicle-class restrictions, guest payment, and status. **Organization** distinguishes owner, operator, and network roles through an ISA mapping. Relations use integer surrogate primary keys; organization subtypes share the parent key. Business identifiers are candidate keys where applicable.

2.2 Ports, Operating Periods, and Adapter Relations

OperatingPeriod replaces unstructured free-text hours with structured daily opening and closing times. **Port** tracks specific station outlets, current types, and power capacities, standardized via **ConnectorType**. **Adapter** handles real-world connector mismatches between plug and socket types by linking separate connector specifications.

*Project repository: https://github.com/chohoon901/PluggedIn.

2.3 Vehicles, Drivers, and Membership Relations

Vehicle stores VIN, plate, make, model, class, range, and maximum charging power; **Driver** stores personal and carrier details. **vehicle_accepts** links vehicles to connector types, and **membership** links drivers to networks with validity dates. Payment-method information belongs to **network**.

2.4 Relationship Mapping and Integrity Constraints

Foreign keys represent many-to-one links. Candidate keys enforce `UNIQUE` and `NOT NULL`; `CHECK` constraints limit current types, coordinates, positive values, and opening periods. SQLite enables foreign keys on every connection, with schema-defined cascade and restriction rules. Application validation handles input formats and aggregate participation, retaining explicitly incomplete source evidence.

3 RELATIONAL OPERATIONS

3.1 Creating and Populating Relations

PluggedIn implements 13 SQLite tables with primary, foreign, candidate-key, and domain constraints. The following SQL defines **Port**:

```
CREATE TABLE port (
    port_id INTEGER PRIMARY KEY,
    station_id INTEGER NOT NULL
        REFERENCES station (station_id) ON DELETE CASCADE,
    number SMALLINT NOT NULL CHECK (number > 0),
    connector_type_id INTEGER NOT NULL
        REFERENCES connector_type (connector_type_id),
    power_kw NUMERIC(6,1) CHECK (power_kw > 0),
    current_type VARCHAR(5)
        CHECK (current_type IN ('AC', 'DC')),
    quantity SMALLINT NOT NULL DEFAULT 1
        CHECK (quantity > 0),
    CONSTRAINT ak_port UNIQUE (station_id, number));
```

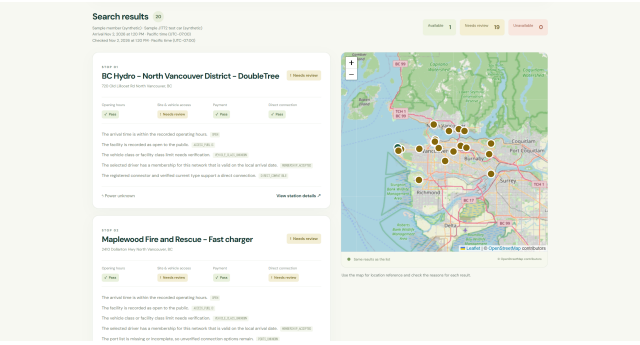


Figure 2: PluggedIn search results with station evaluations and a map.

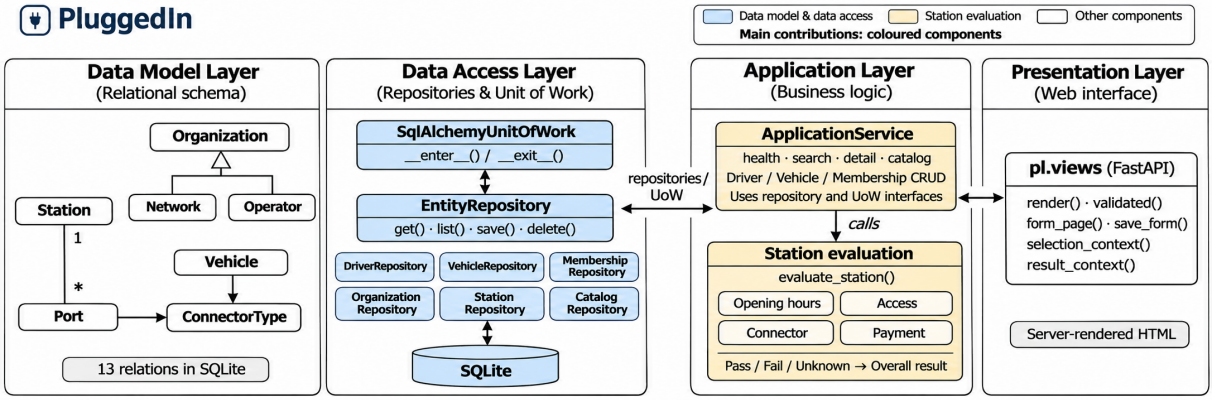


Figure 1: An overview of the PluggedIn system architecture. Coloured components are the system's main contributions.

Foreign keys link ports to stations and connector types. `UNIQUE (station_id, number)` prevents repeated numbers within a station; station deletion cascades to ports.

Sample data is populated using SQLAlchemy ORM, with duplicate checks to maintain data integrity.

3.2 SELECT Queries

The following query finds stations sharing a stored connector ID with Vehicle 2 by joining **Port** and **VehicleAccepts**.

```
SELECT DISTINCT p.station_id
FROM port AS p JOIN vehicle_accepts AS va
    ON p.connector_type_id = va.connector_type_id
WHERE va.vehicle_id = 2;
```

The next query counts compatible port records per station using `COUNT(*)` and `GROUP BY`.

```
SELECT p.station_id, COUNT(*) AS compatible_port_count
FROM port AS p JOIN vehicle_accepts AS va
    ON p.connector_type_id = va.connector_type_id
WHERE va.vehicle_id = 2 GROUP BY p.station_id;
```

On the unchanged sample, both queries return station 1, with one matching port record. Full evaluation additionally interprets connector aliases and checks hours, access, and payment.

3.3 INSERT, UPDATE, and DELETE Operations

The following transaction demonstrates relationship and attribute updates without retaining changes.

```
BEGIN;
INSERT INTO vehicle_accepts (vehicle_id, connector_type_id)
VALUES (1, 1);
UPDATE port SET power_kw = 120.0 WHERE port_id = 1;
DELETE FROM vehicle_accepts
WHERE vehicle_id = 1 AND connector_type_id = 1;
ROLLBACK;
```

These statements temporarily add J1772 support to Vehicle 1, update Port 1, and remove that association. `ROLLBACK` leaves the sample unchanged.

4 ARCHITECTURE AND USER INTERFACE

PluggedIn is a server-rendered FastAPI application using local SQLite. The presentation (`pl`), domain (`cmn`), and data access (`dal`) layers are connected in `main.py`. CRUD operations for drivers, vehicles, and memberships, station searches, and detail pages work without JavaScript. Leaflet displays OpenStreetMap tiles as an optional map.

`cmn` validates input and evaluates hours, access, connectors, and payment as **Pass/Fail/Unknown**, yielding **Available/Unavailable/Needs Review**. `dal` uses SQLAlchemy ORM and bound parameters; a unit of work commits or rolls back related changes per use case. Forms redirect with HTTP 303 on success or return HTTP 400 HTML preserving input; API duplicate keys return HTTP 409.

5 REPRODUCIBILITY AND EVALUATION

The application uses Python 3.12 or later, FastAPI 0.162.0, and SQLAlchemy 2.0.55. `dal/scripts/schema.sql` defines 13 tables and 65 columns. Section 3 describes the main queries; `dal/repositories/` contains ORM implementations, and `README.md` provides setup instructions.

Install `requirements.txt` and the project in a virtual environment, validate the bundled sample with `validate_db`, then start uvicorn. `import_sample` rebuilds data into a fresh directory. UNIQUE and FK constraints enforce integrity. Station owner/operator/network references allow NULL. Network deletion cascades to memberships but may be blocked by referencing stations.

The repository records 170 passing tests, including browser tests, and 91 subtests on 27 October 2026. Team-authored code uses the MIT License; bundled Leaflet 1.9.4 uses BSD 2-Clause, with its full license text included in the repository.

Contributions. Sunghun Cho: Development direction, data integration, usability review, and documentation preparation and review; Jungwon Youn: SQL query design, UI improvements, system architecture diagram, and documentation; Elias Dockal: Database & Relational Model design and upload, code improvements.

AI assistance. OpenAI Codex followed the development plan using a test-first approach. Review identified integer overflow, metadata exceptions, and mobile layout issues, reproduced with failing tests and fixed.

RELATIONS AND CONSTRAINTS

Underlining denotes a primary key (PK). CK denotes an additional candidate key enforced by `UNIQUE` and `NOT NULL`. PKs are generated integers, except for organization subtypes, which share the superclass key. All PK/CK attributes are mandatory. Other attributes are nullable unless marked mandatory below.

(1) **organization** (organization_id, name, website).
CK: {name}. Specialization is overlapping and total: each organization has at least one role.

(2) **site_owner** (organization_id, business_type).
PK/FK: organization_id references organization.organization_id, mandatory. No additional CK.

(3) **operator** (organization_id, support_phone).
PK/FK: organization_id references organization.organization_id, mandatory. No additional CK.

(4) **network** (organization_id, payment_method).
PK/FK: organization_id references organization.organization_id, mandatory. No additional CK.

(5) **station** (station_id, latitude, longitude, name, street, city, province, status, access_restriction, accepts_guest_payment, max_vehicle_class, has_truck_parking, owner_id, operator_id, network_id).
CK: {latitude, longitude}. Latitude is between -90 and 90, and longitude between -180 and 180. FK: owner_id references site_owner.organization_id and operator_id references operator.organization_id, network_id references network.organization_id, all optional. Accepts_guest_payment and has_truck_parking are mandatory Booleans, both defaulting to `FALSE`.

(6) **connector_type** (connector_type_id, name, max_power_kw).
CK: {name}. Max_power_kw must be positive when supplied.

(7) **port** (port_id, station_id, number, connector_type_id, power_kw, current_type, quantity).
CK: {station_id, number}. FK: station_id references station.station_id and connector_type_id references connector_type.connector_type_id, both mandatory. Number must be positive. Quantity is mandatory, positive, and defaults to 1. When supplied, power_kw must be positive and current_type must be `AC` or `DC`.

(8) **operating_period** (operating_period_id, station_id, day, open_time, close_time).
CK: {station_id, day, open_time}. FK: station_id references station.station_id, mandatory. All attributes are mandatory. Day is one of `Mon`, `Tue`, `Wed`, `Thu`, `Fri`, `Sat`, or `Sun`. Close_time must be later than open_time.

(9) **adapter** (adapter_id, model, manufacturer, max_power_kw, from_connector_type_id, to_connector_type_id).
CK: {model}. Both connector attributes are mandatory FKs to connector_type.connector_type_id. The two connector IDs must differ. Max_power_kw must be positive when supplied.

(10) **vehicle** (vehicle_id, vin, licence_plate, make, model, class, range_km, max_charge_kw).
CK: {vin}. Range_km and max_charge_kw must be positive when supplied.

(11) **vehicle_accepts** (vehicle_accepts_id, vehicle_id, connector_type_id).
CK: {vehicle_id, connector_type_id}. FK: vehicle_id references vehicle.vehicle_id and connector_type_id references connector_type.connector_type_id, both mandatory.

(12) **driver** (driver_id, licence_number, name, carrier_name).
CK: {licence_number}.

(13) **membership** (membership_id, driver_id, network_id, start_date, end_date).
CK: {driver_id, network_id, start_date}. FK: driver_id references driver.driver_id and network_id references network.organization_id, both mandatory. End_date is NULL or on or after start_date.

Participation and additional rules. Application checks enforce organization roles and aggregate minimum counts: ports/hours per station, stations per owner/operator, and connectors per vehicle. Explicit `INCOMPLETE` station metadata permits missing port/hour evidence. Overlapping operating periods and memberships remain allowed.

Referential actions. Cascade deletion applies from organizations to subtypes, stations to ports and operating periods, vehicles to vehicle_accepts, and drivers or networks to memberships. Other FKs block deletion while dependent rows remain; these restrictions can also block a cascade.

Implementation boundary. SQLite foreign-key enforcement must be enabled on every connection. Optional numeric attributes allow NULL despite positivity checks. Rejecting empty mandatory text values requires additional validation.